3736. Minimum Moves to Equal Array Elements III

Daily Question - November 3, 2026

Problem Statement

You are given an integer array nums.

In one move, you may increase the value of any single element nums[i] by 1.

Return the minimum total number of moves required so that all elements in nums become equal.

Examples

Example 1: nums = [2,1,3]. Output: 3. Explanation: Increase 2 to 3, then increase 1 to 2 and then to 3. All elements become 3.

Example 2: nums = [4,4,5]. Output: 2. Explanation: Increase both 4s to 5.

Constraints

1 <= nums.length <= 100; 1 <= nums[i] <= 100.

Key Insight

A move can only increase one element by 1. Therefore, no element can be decreased, and the final common value must be at least the current maximum. The minimum target is exactly max(nums), because raising every element to the maximum requires no unnecessary extra moves.

For each element, the required moves are max(nums) - nums[i]. Thus the total is:

answer = max(nums) * n - sum(nums)

Complete Java Solution

```
class Solution {
    public int minMoves(int[] nums) {
        int maximum = Integer.MIN_VALUE;
        int sum = 0;

        for (int value : nums) {
            maximum = Math.max(maximum, value);
            sum += value;
        }

        return maximum * nums.length - sum;
    }
}
```

Correctness

Because every move increases an element, the final equal value must be at least the original maximum. Choosing the original maximum as the target gives the smallest possible final value and therefore the smallest total number of increments. Each element nums[i] must be increased by exactly max(nums) - nums[i], and these increments are independent, so their sum is both necessary and sufficient.

Complexity

Time: O(n), since the array is scanned once. Auxiliary space: O(1), because only scalar variables are used.

Implementation Notes

With n <= 100 and nums[i] <= 100, maximum * n - sum is at most 100 * 100 - 1 = 9999, so int arithmetic is safe. If all elements are already equal, the formula returns 0.